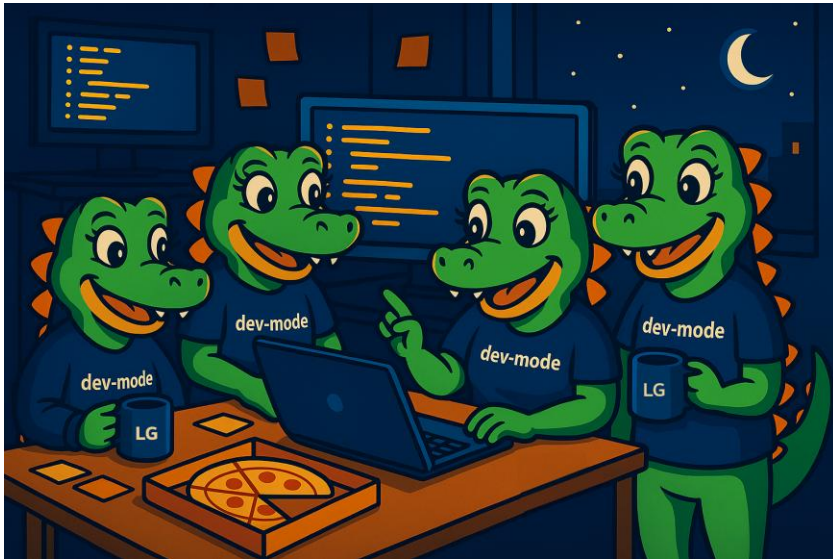




# LATER GATOR

## SPRINT 2 RETROSPECTIVE

### PREPARED FOR

CEN3031 – Introduction to Software Engineering  
FALL 2025 | University of Florida  
Professor Lisha Zhou

### PREPARED BY

TEAM LATER GATOR:  
Danielle Foege  
Madison Holt  
Carrie Ruble  
Samantha Wong

## Insights Activity

---

During this sprint, Team Later Gator encountered significantly more technical complexity than anticipated, which led to a significant restructuring effort. Our original plan was to integrate a Room database into our React Native project, but we quickly learned that Room depended on KSP/KAPT and generated Kotlin files that were incompatible with the React Native environment. This mismatch caused persistent build failures, unresolved references, and broken imports that blocked our progress for several days.

Instead of staying stuck, the team used this challenge as an opportunity to learn and adapt. After examining the underlying issues, we collaboratively decided to remove Room entirely and pivot to a simpler, more reliable solution using a pre-populated SQLite database. This pivot restored our ability to build and launch the app, and it allowed us to continue implementing features without relying on fragile or incompatible tools.

The insights generated during this activity centered on tool evaluation, problem-solving strategies, communication, and how our team supported each other during a demanding sprint. We reflected on how the technical roadblocks shaped our workflow and recognized the importance of identifying blockers early, sharing information consistently, and staying flexible when plans need to change.

### Event Timeline

This sprint covered the period from October 30th to November 15th, 2025, and it included several major turning points for the project. Early in the sprint, the team focused on setup tasks and initial feature work, expecting to continue building on the foundation established during Sprint 1. However, as soon as we attempted to integrate Room into our React Native project, the timeline shifted sharply. What began as routine implementation work quickly turned into a multi-day investigation into build errors, KSP/KAPT conflicts, Kotlin metadata mismatches, and breaking interactions between Room and React Native.

As shown in Figure 1, our timeline captured this dramatic shift clearly. The sticky notes in the early portion reflect steady progress and optimism, while the cluster of notes in the middle of the timeline shows the repeated technical setbacks that slowed development. Ultimately, the team agreed on a major pivot: removing Room entirely and switching to a pre-populated SQLite database. This decision, represented near the end of the timeline, marked the moment where progress resumed and the team regained momentum.

By the end of the sprint, the team had not only rebuilt the technical foundation of the project but had also learned how to navigate high-stakes technical decisions under pressure. Emotionally, the timeline reflected a progression from optimism, to frustration, to relief and renewed momentum once the pivot succeeded. Despite the challenges, the team stayed engaged, supportive, and focused on finding workable solutions.



Figure 1: Team Later Gator visualizing key sprint events using a timeline of successes, challenges, and milestones.

## Select/Group Insights

After completing the timeline, the team moved into grouping our individual insights into broader themes. This activity helped us visualize not just the sequence of events, but the underlying patterns behind the challenges we encountered this sprint. As shown in Figure 2, we began by sorting each sticky note into swimlanes that represented categories such as technical hurdles, communication, workflow, tool selection, and task management.

This grouping process made it clear that many of our insights were interconnected. In particular, several notes clustered around the difficulties we faced with integrating Room into a React Native environment. These notes reflected confusion, repeated build failures, and the time spent trying to diagnose problems that stemmed from incompatible tools. Other clusters highlighted positive aspects of the sprint, such as strong team communication, effective pair programming when issues became overwhelming, and a shared willingness to adapt when our original plan stopped working.



Figure 2: Team members collaboratively sorting insights into swimlanes for clearer categorization.



The final groupings, shown in **Figure 3**, revealed three dominant themes:

1. Technical Adaptation: recognizing when a tool is creating more problems than it solves, and being willing to pivot.
2. Communication and Transparency: keeping the team informed when blockers arise so others can assist or adjust.
3. Effective Workflows: breaking tasks into smaller components, testing earlier, and validating technical assumptions before committing.

These groupings helped set the stage for both our dot voting and the action items we developed for the next sprint.

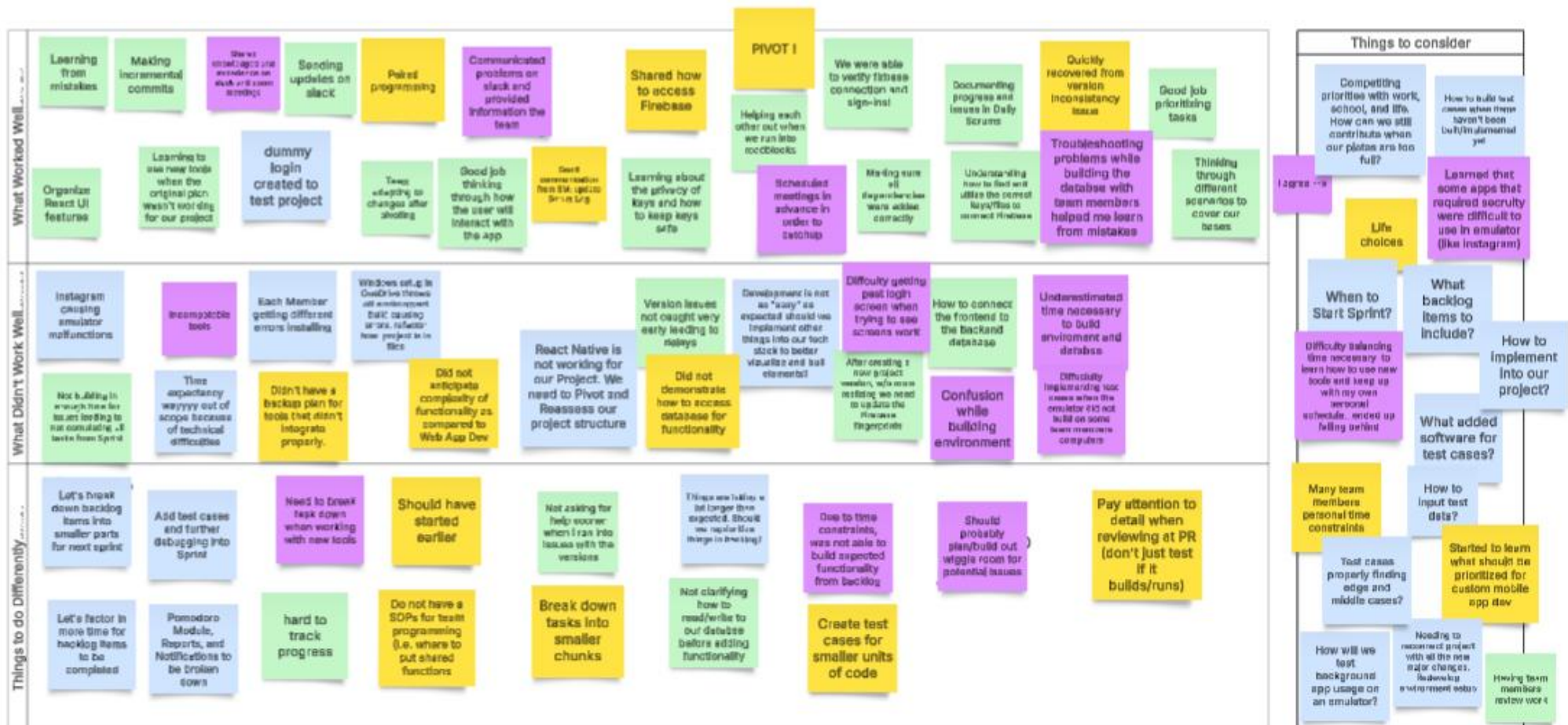


Figure 3: Final groupings of insights showing primary categories for team growth and improvement.

## Vote on Insights

Once the insights were grouped, each team member participated in a silent dot-voting process to identify which issues had the greatest impact on the sprint. As shown in Figure 4, every team member received five votes to distribute across the insights they felt were most important to address moving forward. This allowed us to prioritize without discussion influencing individual perspectives.

The voting revealed a clear pattern: the most heavily voted insights centered on the technical obstacles we encountered while attempting to integrate Room. Notes related to KSP/KAPT conflicts, repeated build failures, and overwhelming time spent debugging rose to the top quickly. Closely following were insights about communication, particularly the need to share blockers earlier, post updates more consistently in Slack, and keep the team aware of progress and setbacks in real time.

Other insights earned votes for their focus on workflow improvements, including breaking large tasks into smaller ones, testing new additions more frequently, and validating tool compatibility earlier in the development process. Overall, the dot voting helped the team align on which areas need the most attention in the next sprint and ensured that each member's perspective contributed equally to setting priorities.

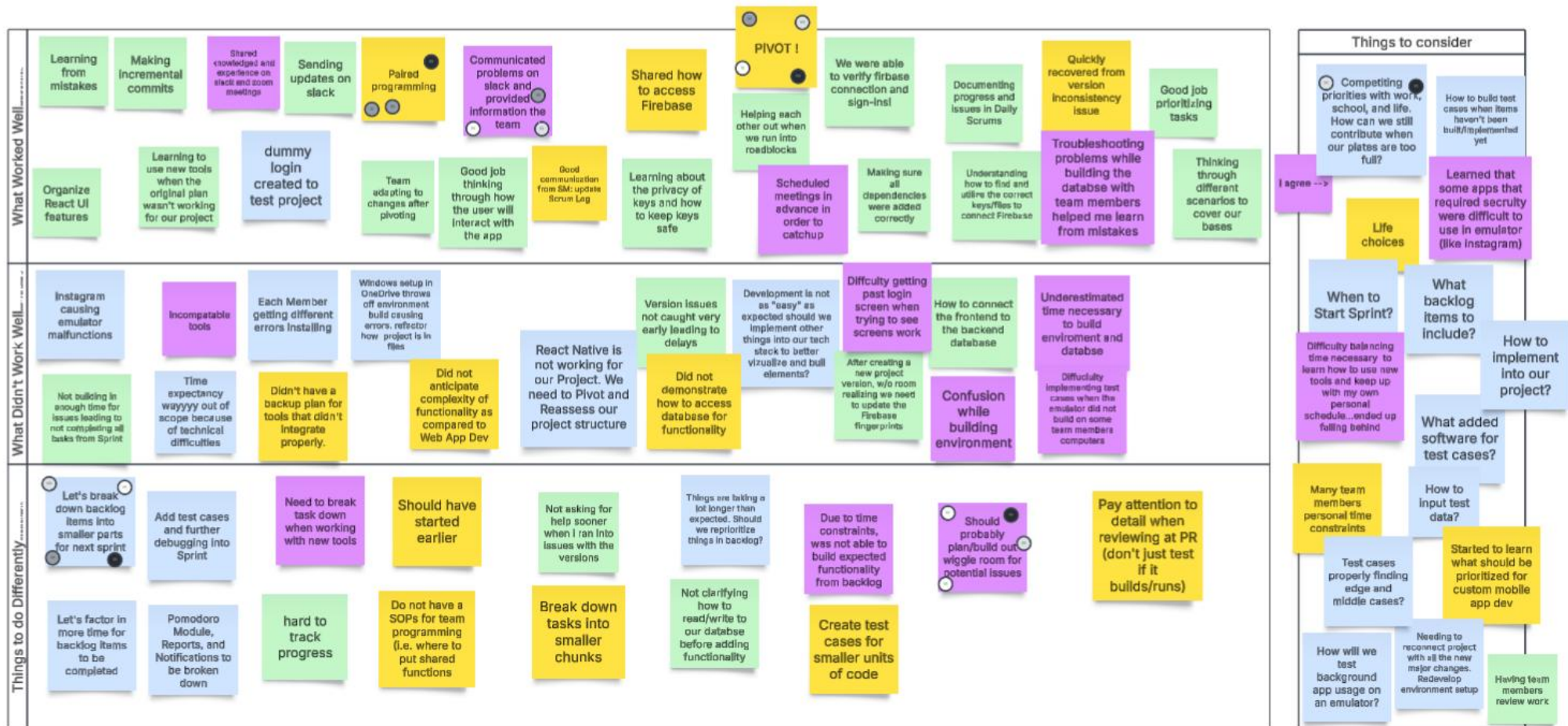


Figure 4: Team members using dot voting to prioritize insights based on collective agreement.

## Decide on Actions

After prioritizing the insights through dot voting, the team discussed the highest-voted items and translate them into clear, actionable steps for the next sprint. Unlike our previous retrospective, the insights in Figure 5 were not arranged in any specific order beyond simply being placed on the board for discussion. Their placement does not represent priority or categorization, this time, the focus was on conversation and clarity rather than visual structure.

During the discussion, several consistent themes emerged. The most prominent was the need to validate tool and framework choices earlier in the development cycle. Our experience with Room underscored how quickly incompatible technologies can halt progress, so the team agreed that future sprints should include more upfront research, earlier testing, and faster identification of technical blockers.

We also recognized the importance of clearer communication. Several action items center on keeping Slack updated, reporting blockers sooner, and making sure team members are aware of ongoing issues so they can offer support before delays accumulate. Workflow improvements were another major focus, breaking tasks into smaller pieces, testing new additions more frequently, and maintaining shared visibility into individual progress.

Finally, the team reinforced the value of staying flexible. The decision to pivot away from Room and rebuild the database layer using SQLite demonstrated how adaptability can save a sprint. Throughout the process, the team remained patient and compassionate with one another, recognizing that none of us have prior experience with mobile development or many of the tools we are learning for the first time. Instead of becoming discouraged, we supported each other's learning curves and took pride in how quickly we were able to regroup, pivot, and move forward after such a significant technical setback. This mindset is something we intend to carry into future sprints. save a sprint. Moving forward, we aim to remain open to new approaches when challenges arise, even if they require shifting direction late in the process.

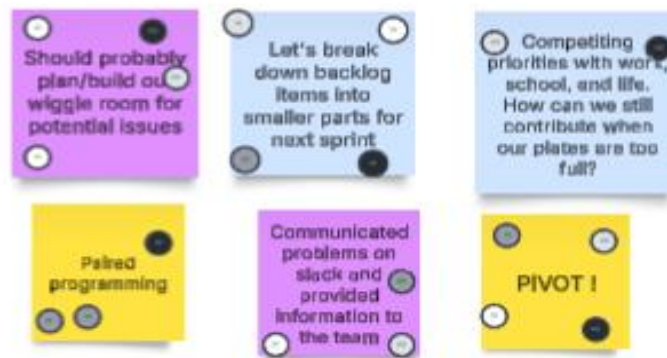


Figure 5: Insights arranged by number of votes to determine next sprint priorities and action items.



## SMAR(T) Plan

---

Our SMAR(T) Plan for the next sprint builds on both the strengths and challenges identified during this retrospective. Many of the goals from Sprint 1 remain relevant, but this sprint's technical hurdles (particularly the extensive restructuring required after discontinuing Room) highlighted the need for more intentional planning, clearer communication, and increased buffer time for learning new tools. The action items below reflect team-wide contributions and represent a shared commitment to staying adaptable, working collaboratively, and supporting one another as we continue developing our prototype.

### Plan Wiggle Room for Potential Issues

- Incorporate time to learn and research how to use new tools.
- Break down tasks into smaller parts and allocate a greater amount of time for each part.
- Continue to look ahead for upcoming assignments and deadlines, but build in more time for each backlog item.
- Communicate individual schedules to help coordinate team meetings and planned Slack check-ins.
- Continue raising problems in Slack so the team can stay aware of issues and get ahead of them.
- Assume worst-case scenarios when estimating tasks so goals remain achievable.

### Paired Programming

- When working on major implementations, have a team member available to help work through complex tasks.
- Working together help understand mistakes and prevent time wasted in confusion
- Don't be afraid to communicate / ask for help

### Break Down Items into Smaller Parts

- Backlog items should have clear boundaries so that they can easily be checked off.
- Define the scope of each task at the time it is created to eliminate ambiguity.
- Make feature additions small and independent of other implementation aspects so that issues in one area do not block unrelated work.
- Test new additions frequently and thoroughly.

## Managing Life and Competing Priorities

- Continue to let our team members know when we are facing difficulties or have packed schedules.
- Plan to work ahead whenever possible but ask for help when approaching a deadline.
- With smaller and more manageable backlog items, ensure there are tasks available for team members with lower bandwidth.

## Communicating Issues in Slack and Providing Information to the Team

- Even when tasks seem straightforward, update the team about what we are working on. This keeps everyone informed of challenges and encourages knowledge sharing.
- When progress is made, post updates so team members can review changes and stay aligned.
- Maintain consistent check-ins so problems surface earlier, preventing future delays.

## PIVOT

- When facing a difficult or seemingly impossible task, pivot to a solution that is simpler and more manageable.
- Start by working from something we know or shift to a more familiar approach when appropriate.
- Try a new solution if the current one is not working, even if that means adjusting the plan late in the sprint.
- Do not feel locked into a specific format—flexibility is key for steady progress.

Together, these goals reflect our shared commitment to growing both technically and collaboratively as we continue building Later Gator. This sprint tested our ability to adapt under pressure, and the team responded with patience, creativity, and mutual support. By carrying these SMAR(T) action items into the next sprint, we aim to strengthen our workflow, stay flexible when facing unexpected challenges, and build on the momentum we regained after our major pivot.